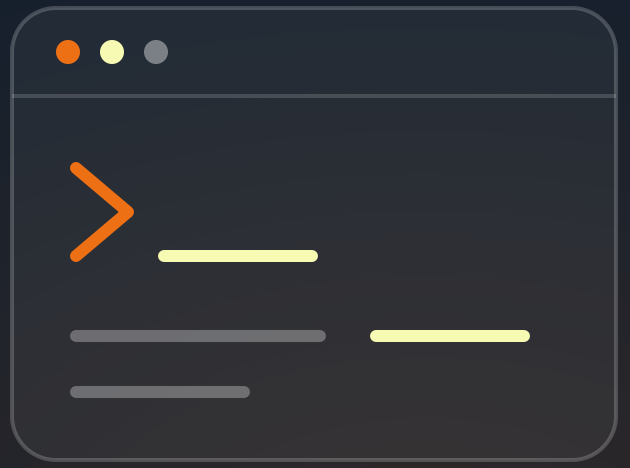
pontIA

MÓDULO

Fundamentos de Linux y terminal

pontIA



Máster en Data Engineering, MLOps & Arquitectura Cloud

2025

¿Quién soy?

Encantado de conoceros :)

pontIA

Estudié en Ingeniería Informática en la Universidad de Salamanca, una vez terminé me especialicé en Data Science y posteriormente en Bioinformática / Genética Computacional.

Actualmente me encuentro desempeñando el rol de Lead Data Scientist dentro del equipo de Data e Inteligencia Artificial de Decathlon España, donde me encuentro desde 2016, habiendo liderado y colaborado en multitud de proyectos tanto a nivel local como internacional.

Además, disfruto como docente en pontia.tech dentro de algunos de los másteres además de dirigir el Máster en Data Engineering, MLOps y Arquitectura Cloud.

Gracias a ello, puedo disfrutar combinando algunas de mis pasiones: utilizar datos para tomar decisiones, ayudar a disponibilizar la IA para extender la capacidad de las personas y, además, hacerlo sobre algo que me encanta como es el deporte.

PontIA



Sergio Benito Martín

Lead Data Scientist

@sergio-benito pontia@sergiobenito.com

Evaluación: práctica + test

Entrega individual

pontIA

Cómo se calcula

60% Mini-retos prácticos realizados durante la sesión en vivo.

40% Test posterior sobre fundamentos, comandos y diagnóstico de errores.

Condiciones

Entrega de los comandos y de las evidencias o ficheros solicitados en un documento de tipo PDF donde se muestren tanto los comandos escritos como la salida a nivel de captura de pantalla.

pontIA

La terminal será el punto de entrada a casi todo

Un mismo hábito conecta herramientas muy distintas



Si entiendes rutas, procesos, variables y errores, las nuevas herramientas dejan de parecer cajas negras.

El objetivo es desenvolverse, no recitar comandos

Al terminar podrás completar un pequeño flujo de trabajo sin interfaz gráfica

pontIA

01	**Orientarte y moverte**	por un sistema de archivos
02	**Crear, copiar, mover y borrar**	con seguridad
03	**Inspeccionar datos**	y buscar problemas en logs
04	**Combinar comandos**	y guardar resultados
05	**Ejecutar scripts**	y diagnosticar errores frecuentes

pontIA

Contenido

0 Preparación del entorno

1 Entender la terminal

2 Navegar y usar rutas

3 Trabajar con archivos

4 Inspeccionar y buscar

5 Combinar comandos

6 Ejecutar y diagnosticar

pontIA

0

Preparar el entorno

Una instalación estable hoy evita incidencias en Docker, Airflow, Spark y cloud más adelante.

Comparativa: Tres experiencias de terminal, una misma meta

pontIA

La elección importa menos que comprender rutas, procesos, permisos y automatización.

	Linux	macOS	Windows
Shell habitual	Bash	zsh	PowerShell & WSL
Rutas	/home/sergio	/Users/sergio	C:\Users\… / & /home/…
Instalar paquetes	apt	brew*	winget / apt en WSL
Uso Linux	Nativa	Muy próxima	Muy próxima con WSL
Curva inicial	Media	Baja-media	Media por doble entorno

* Homebrew no viene instalado por defecto.

¿Por qué Linux es la referencia en terminal?

Porque reduce la distancia entre aprender, automatizar y operar software real.

pontIA

01	**Servidores y cloud**	El entorno objetivo suele ser Linux; aprenderlo evita traducciones.
02	**Automatización**	Shell scripts, pipes, SSH y tareas programadas son parte del flujo habitual.
03	**Contenedores**	Docker Engine se instala de forma nativa en distribuciones Linux.
04	**Ecosistema abierto**	Herramientas, documentación y código pueden inspeccionarse y combinarse.
05	**Eficiencia**	Administrar por texto consume pocos recursos y funciona bien de forma remota.

pontIA

Para esta asignatura, todos trabajaremos en un entorno Linux

El sistema de tu ordenador puede ser distinto; el lenguaje común será Bash y las utilidades GNU/Linux.

pontIA

SI USAS LINUX

Terminal nativa

La ruta más directa.

SI USAS MACOS

Terminal + zsh

Casi todo será equivalente; señalaremos diferencias.

SI USAS WINDOWS

WSL 2 + Ubuntu

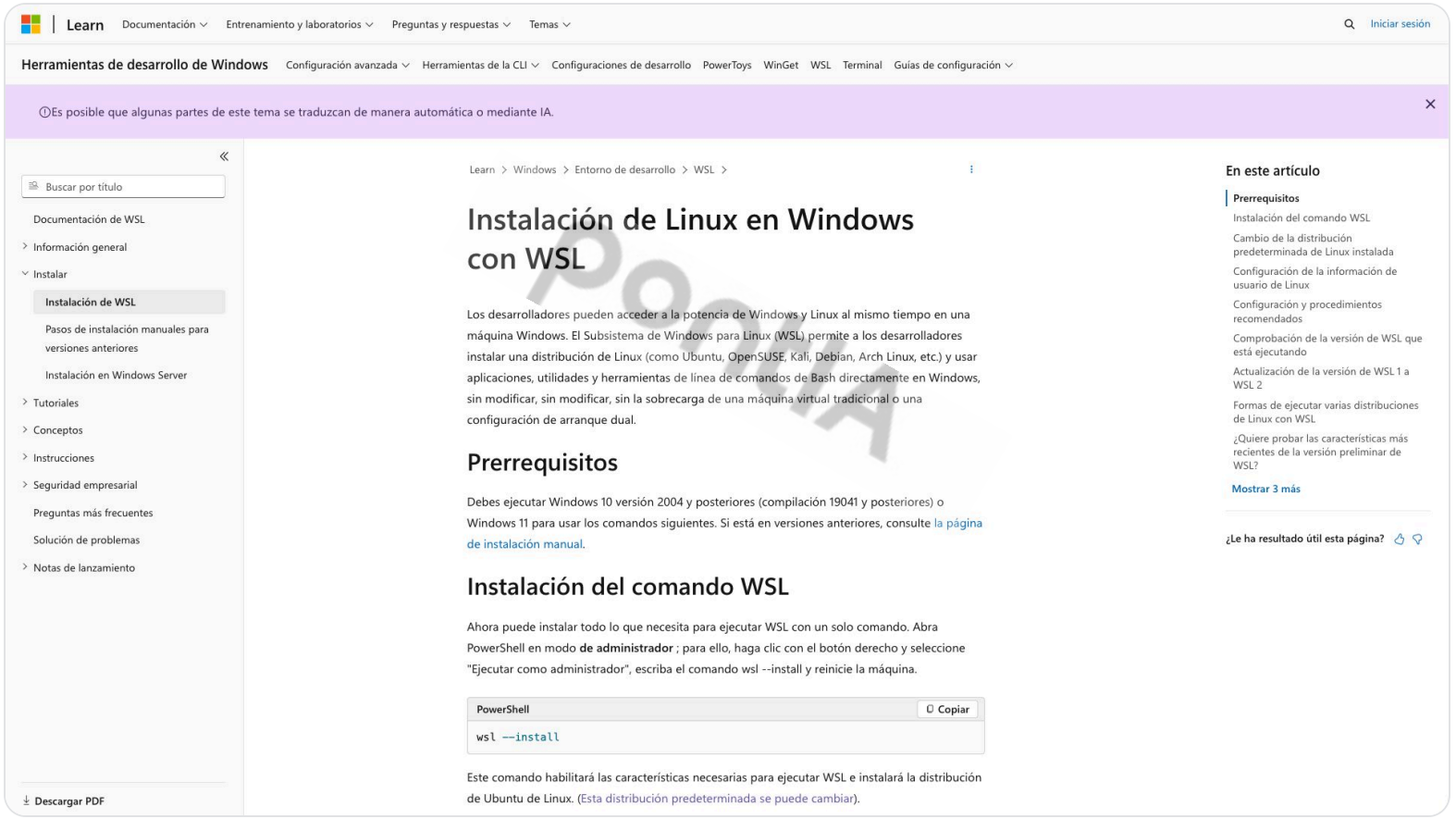
Trabajarás con Linux sin abandonar Windows.

Objetivo: que el mismo comando funcione para toda la clase.

Windows permite instalar Linux de manera sencilla

Seguiremos la guía de instalación de Windows

pontIA



Windows: instala WSL 2 con Ubuntu

Ejecuta la instalación una sola vez desde PowerShell como administrador

pontIA

PowerShell (Administrador)

```
PS> wsl --install -d Ubuntu

# Reinicia Windows si se solicita
PS> wsl --list --verbose
```

Debes ver

```
Ubuntu
STATE: Running o Stopped
VERSION: 2
```

Después crea tu usuario y contraseña de Linux.

Windows: instala WSL 2 con Ubuntu

PowerShell (Administrador)

```
# Primero instalamos WSL y reiniciamos
PS> wsl --install

# Configuramos para utilizar la versión 2 de WSL
PS> wsl --set-default-version 2
```

pontIA

```
PS C:\Users\Sergio> wsl --set-default-version 2
Para obtener información sobre las diferencias clave con WSL 2, visita https://aka.ms/wsl2

La operación se completó correctamente.
PS C:\Users\Sergio>
```

Windows: instala WSL 2 con Ubuntu

PowerShell (Administrador)

```
# Reinicia Windows si se solicita
# Y muestra las versiones de Linux disponibles
PS> wsl --list --verbose
```

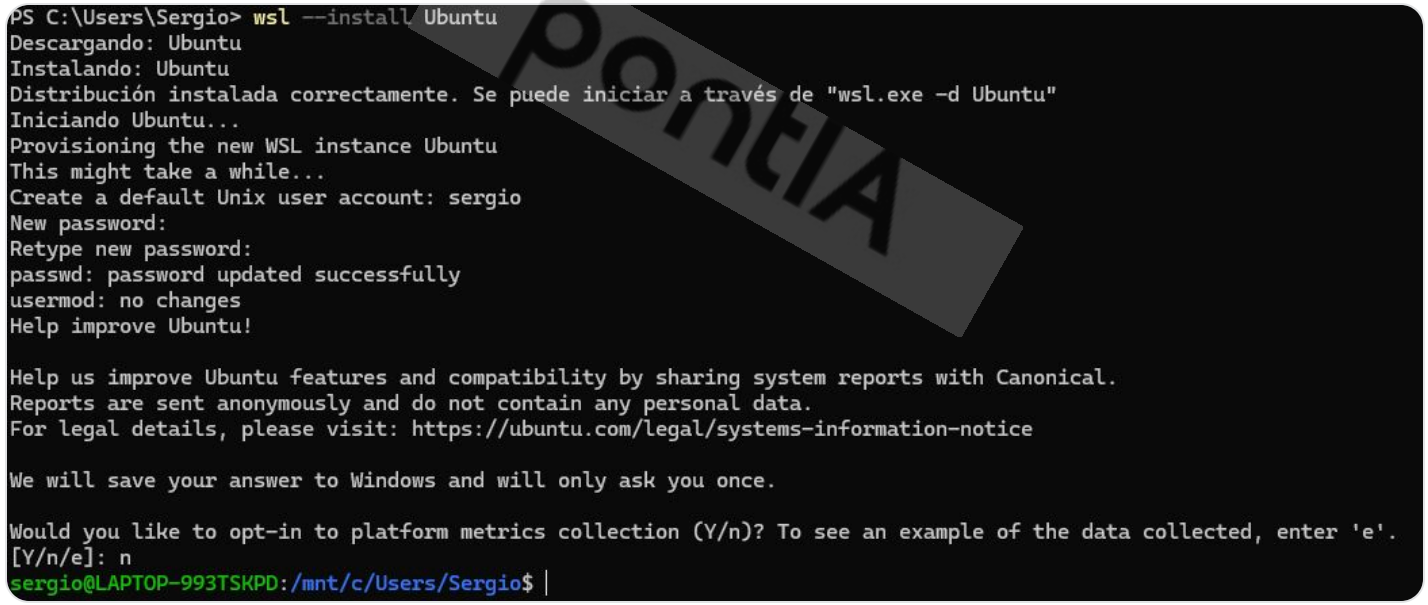
```
PS C:\Users\Sergio> wsl --list --online
A continuación se muestra una lista de distribuciones válidas que se pueden instalar.
Instalar con "wsl.exe --install <Distro>".

NAME                            FRIENDLY NAME
Ubuntu                          Ubuntu
Ubuntu-26.04                    Ubuntu 26.04 LTS
Ubuntu-24.04                    Ubuntu 24.04 LTS
Ubuntu-22.04                    Ubuntu 22.04 LTS
openSUSE-Tumbleweed             openSUSE Tumbleweed
openSUSE-Leap-16.0              openSUSE Leap 16.0
SUSE-Linux-Enterprise-15-SP7    SUSE Linux Enterprise 15 SP7
SUSE-Linux-Enterprise-16.0      SUSE Linux Enterprise 16.0
kali-linux                      Kali Linux Rolling
Debian                          Debian GNU/Linux
AlmaLinux-8                     AlmaLinux OS 8
AlmaLinux-9                     AlmaLinux OS 9
AlmaLinux-Kitten-10             AlmaLinux OS Kitten 10
AlmaLinux-10                    AlmaLinux OS 10
archlinux                       Arch Linux
FedoraLinux-44                  Fedora Linux 44
FedoraLinux-43                  Fedora Linux 43
eLxr                            eLxr 12.12.0.0 GNU/Linux
OracleLinux_7_9                 Oracle Linux 7.9
OracleLinux_8_10                Oracle Linux 8.10
OracleLinux_9_5                 Oracle Linux 9.5
SUSE-Linux-Enterprise-15-SP6    SUSE Linux Enterprise 15 SP6
```

Windows: instala WSL 2 con Ubuntu

PowerShell (Administrador)

```
# Ahora instalamos Ubuntu
PS> wsl --install -d Ubuntu
```



Windows: WSL trabaja dentro del sistema de archivos Linux

La ubicación recomendada para trabajar es tu carpeta personal

pontIA

Ubuntu / WSL

```
# Ir a la home del usuario
$ cd ~

# Verificar dónde se está trabajando
$ pwd
/home/sergio

# Crea una carpeta anidada
$ mkdir -p master/linux-terminal

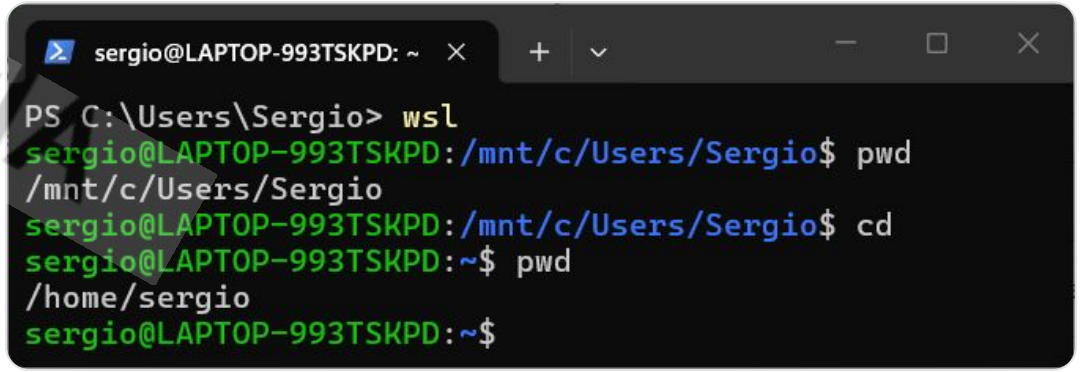
# Muévete a ella
$ cd master/linux-terminal
```

RECOMENDADO

/home/usuario/proyecto

EVITA PARA EL TRABAJO HABITUAL

/mnt/c/Users/...



macOS: Terminal y zsh ya vienen preparados

No necesitas instalar Linux para poder trabajar

pontIA

Terminal (macOS)

```
# Instalar herramientas básicas desarrollo (si no están ya instaladas)
% xcode-select --install

% echo $SHELL
/bin/zsh

% uname -s
Darwin

% python3 --version
Python 3.13.6
```

pontIA

- Abre Terminal desde Aplicaciones
- Trabaja dentro de tu carpeta personal
- Usa los mismos materiales ZIP que el resto

1

Entender la terminal

Cuando distingues terminal, shell y sistema operativo, los mensajes dejan de ser misteriosos.

La terminal muestra; la shell interpreta

Diferencia entre términos que se confunden habitualmente

pontIA

Terminal, shell y kernel

Tres capas, una misma interacción

comandos ↓

1. TERMINAL
La ventana donde escribes y ves resultados

2. SHELL
Interpreta tus comandos y pide acciones al sistema

3. KERNEL
El núcleo que gestiona hardware y recursos

↑ resultado

`ls` → shell lo interpreta → kernel accede al sistema de archivos

La terminal muestra; la shell interpreta

Linux o macOS ejecutan finalmente la operación

pontIA



Ubuntu suele usar Bash · macOS usa zsh por defecto

Un comando combina acción, opciones y argumentos

Lee de izquierda a derecha antes de ejecutarlo

Ejemplo

```
$ ls -lah data/
```

pontIA

comando
qué hacer

opciones
cómo hacerlo

argumento
sobre qué

El prompt te dice dónde y con quién estás

El símbolo $ o % no forma parte del comando

pontIA

Prompt

```
sergio@ubuntu:~/master$ pwd
/home/sergio/master
```

- **sergio**: usuario
- **ubuntu**: equipo o entorno
- **~/master**: directorio actual
- **$ o %**: listo para recibir un comando

Consejo: mira siempre el directorio antes de copiar un comando.

La productividad empieza con cuatro gestos

Usa la shell como una herramienta interactiva

pontIA

`Tab`	Autocompleta nombres y descubre coincidencias
`↑ / ↓`	Recorre el historial de comandos
`Ctrl+C`	Interrumpe la mayoría de procesos en primer plano
`clear`	Limpia la vista sin borrar el historial
`man / --help`	Consulta la ayuda disponible localmente

pontIA

2

Navegar y usar rutas

La terminal siempre opera desde un lugar concreto; saber dónde estás evita la mayoría de errores iniciales.

El sistema de archivos es un árbol

Todo parte de / y tu espacio habitual comienza en ~

pontIA

Vista simplificada

```
/
├── home
│   └── sergio          ← ~ ó $HOME
│       └── master
│           └── linux-terminal
├── etc
├── tmp
└── var
```

EN MACOS

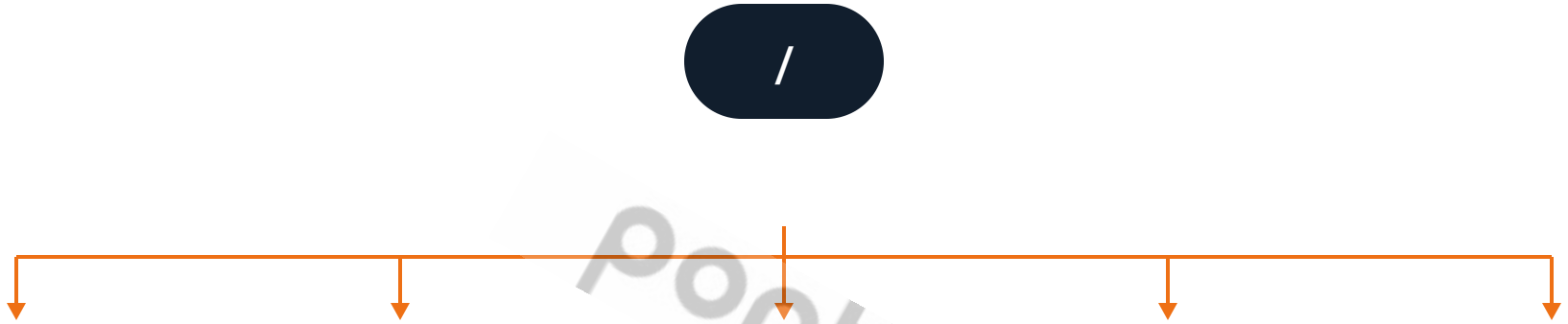
Tu home suele estar en:

`/Users/sergio`

No necesitas memorizar todo el árbol. Empieza por reconocer `/`, `~` y tu proyecto.

El sistema de archivos es un árbol

Todo parte de / y tu espacio habitual comienza en ~



pontIA

/home	/etc	/var	/tmp	/usr
datos de usuarios	configuración	datos variables y logs	temporales	programas y bibliotecas

Una ruta absoluta empieza en /. Una ruta relativa empieza en tu directorio de trabajo actual.

Primero ubícate; después explora

pwd responde 'dónde estoy' y ls responde 'qué hay aquí'

pontIA

Orientación

```
$ pwd
/home/sergio/master/linux-terminal

$ ls
data          logs          scripts       README.md

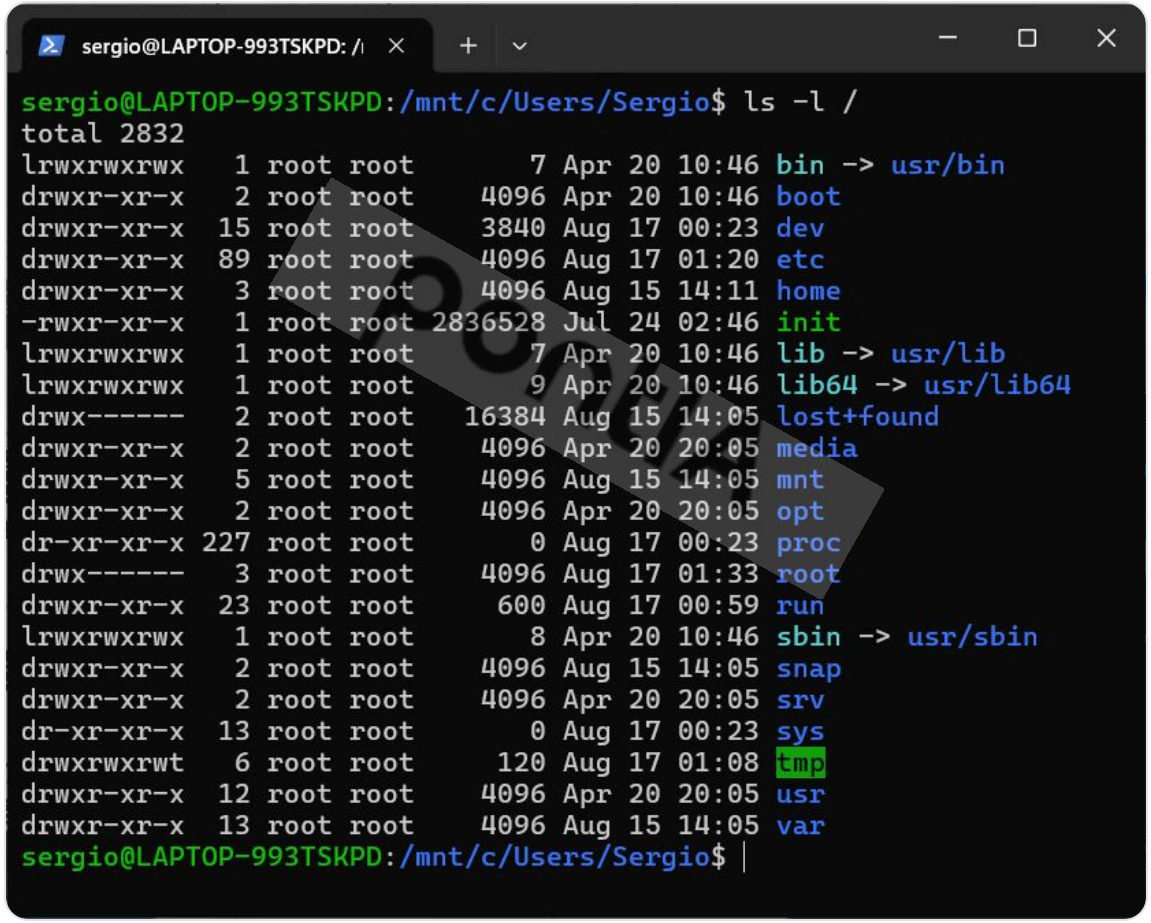
$ ls -lah
```

`pwd`	Directorio actual
`ls`	Contenido
`ls -l`	Detalles
`ls -a`	Incluye ocultos

Qué sucede si lanzamos el comando ls -l del directorio raíz

De esta manera podremos ver todas las carpetas clave de nuestro sistema operativo

pontIA



cd cambia el contexto de los siguientes comandos

Muévete paso a paso y verifica cuando tengas dudas

pontIA

Cambiar de directorio

```
$ cd data
$ pwd

$ cd ..
$ cd scripts

$ cd ~
$ cd -
```

`cd carpeta`	Se mueve a carpeta
`cd ..`	Se mueve a la carpeta superior
`cd ~`	Se mueve a tu home
`cd -`	Regresa al directorio anterior

⚠ **IMPORTANTE:** Linux es “case sensitive”, es decir, distingue entre minúsculas y mayúsculas

💡 **RECOMENDACIÓN:** nombres siempre en minúsculas, sin símbolos y sin espacios

Una ruta absoluta no depende de dónde estés

Una ruta relativa se interpreta desde el directorio actual

pontIA

Ruta absoluta

```
/home/sergio/master/data/orders.csv
```

- Empieza por /
- Siempre apunta al mismo lugar

Ruta relativa

```
data/orders.csv
```

- Parte del directorio actual
- Es más corta y portable

pontIA

Cuatro símbolos resuelven gran parte de la navegación

Apréndelos como relaciones, no como abreviaturas

pontIA

`/`	Raíz del sistema o separador entre directorios
`~`	Carpeta personal del usuario actual
`.`	Directorio actual
`..`	Directorio padre
`./script.sh`	Archivo script.sh que está aquí

pontIA

3

Trabajar con archivos

Crear, copiar y mover es sencillo; hacerlo sin perder información requiere método.

mkdir crea estructura; touch crea archivos vacíos

Usa -p cuando quieras construir varios niveles

pontIA

Crear

```
$ mkdir output
$ mkdir -p archive/2026/08

$ touch output/report.txt

$ ls -R output archive
```

mkdir -p

Crea los directorios intermedios y no falla si ya existen.

`touch` no es un editor: crea un archivo comienza vacío.

cp duplica; mv cambia la ubicación o el nombre

Comprueba origen y destino antes de pulsar Enter

pontIA

Copiar

- cp origen destino
- cp -r carpeta destino
- El original permanece

Mover o renombrar

- mv origen destino
- Sirve para archivos y carpetas
- El original deja de estar en su ubicación

Ejemplos

```
$ cp data/orders.csv archive/orders.csv
$ mv output/report.txt output/report_old.txt
```

rm elimina sin pasar por la papelera

La seguridad depende de revisar el alcance antes de ejecutar

pontIA

Borrado consciente

```
$ pwd
$ ls output/

$ rm output/report_old.txt

# Carpetas: solo si es realmente necesario
$ rm -r output_old/
```

⚠ Nunca copies a ciegas

- rm -r con rutas amplias
- Comandos con *
- sudo rm ...
- Una ruta que no has verificado

Las comillas protegen espacios; los comodines seleccionan patrones

La shell expande * y ? antes de ejecutar el comando

pontIA

Nombres y patrones

```
$ mkdir "datos históricos"

$ cp "ventas agosto.csv" archive/

$ ls data/*.csv

$ ls logs/pipeline?.log
```

`*`	cero o más caracteres
`?`	exactamente un carácter

⚠ Antes de borrar con un patrón, pruébalo con `ls`.

Los archivos ocultos empiezan por punto

Suelen guardar configuración; no son invisibles para la shell

pontIA

Archivos ocultos

```
$ ls
data          logs          scripts

$ ls -a
.  ..  .env.example  data  logs  scripts
```

.env

Puede contener configuración o secretos. No lo compartas ni lo sobrescribas sin entenderlo.

En el máster verás también `.gitignore`, `.env` y carpetas de configuración.

Vi y Nano: dos editores, dos enfoques

Nano prioriza la sencillez; Vi prioriza la velocidad y el control

pontIA

Nano

- Escritura directa: un único modo
- Atajos visibles con Ctrl
- Ideal para empezar y editar rápido

Vi

- Editor modal: Normal e Inserción
- Muy rápido cuando aprendes comandos
- Disponible en casi cualquier Unix

pontIA

Abrir un archivo

```
$ nano notas.txt
$ vi notas.txt
```

Nano funciona como un editor de texto convencional

Abres, escribes y utilizas atajos con Ctrl

pontIA

Nano

```
$ nano notas.txt
Escribe directamente en el archivo

Ctrl+O   guardar · Enter confirmar
Ctrl+X   salir
```

Flujo básico

1. Abre
2. Escribe
3. Guarda
4. Sal

En Nano, `^` significa Ctrl: `^O` = Ctrl+O.

Vi funciona con modos: primero eliges la acción

Pulsa i para escribir; Esc para volver al modo Normal

pontIA

Vi

```
$ vi notas.txt
i        entrar en Inserción
Esc      volver a Normal

:w       guardar · :q salir · :wq ambos
```

Modos

Normal: navegar y lanzar órdenes.

Inserción: escribir texto.

⚠ `:q!` sale sin guardar los cambios.

4

Inspeccionar y buscar

Antes de procesar datos, comprueba qué archivo tienes y qué contiene realmente.

cat muestra; less permite explorar

Elige según el tamaño del archivo

pontIA

cat

- Archivo corto
- Salida completa
- Útil para combinar con pipes

less

- Archivo largo
- Navegación con flechas o espacio
- q para salir; /texto para buscar

Lectura

```
$ cat config/pipeline.env
$ less logs/pipeline.log
```

head y tail reducen el archivo a una muestra útil

No necesitas imprimir miles de líneas para entender la estructura

pontIA

Muestreo

```
$ head -n 5 data/orders.csv
order_id,date,status,amount
1001,2026-08-01,paid,39.90
...

$ tail -n 20 logs/pipeline.log
$ tail -f logs/pipeline.log
```

`tail -f`

Sigue un archivo mientras crece. Detén la observación con Ctrl+C.

file identifica el tipo; wc cuantifica

Dos comprobaciones baratas antes de analizar

pontIA

`file archivo`	Detecta el tipo de contenido
`wc -l archivo`	Cuenta líneas
`wc -w archivo`	Cuenta palabras
`wc -c archivo`	Cuenta bytes

pontIA

Comprobaciones

```
$ file data/orders.csv
$ wc -l data/orders.csv
```

grep encuentra líneas que contienen un patrón

Empieza simple y añade opciones solo cuando aporten algo

pontIA

Buscar contenido

```
$ grep "ERROR" logs/pipeline.log
$ grep -i "warning" logs/pipeline.log
$ grep -n "order_id=1042" logs/pipeline.log
$ grep -c "ERROR" logs/pipeline.log
```

`-i`	Ignora mayúsculas
`-n`	Muestra número de línea
`-c`	Cuenta coincidencias
`-v`	Excluye coincidencias

find busca nombres y tipos dentro del árbol

Define un punto de partida y un criterio

pontIA

Buscar archivos

```
$ find . -name "*.csv"
$ find logs -type f -name "*.log"
$ find data -type f
```

find

1. Dónde empezar
2. Qué tipo buscar
3. Qué nombre debe cumplir

Pon `*.csv` entre comillas para que la shell no lo expanda antes.

La terminal sirve para inspeccionar, no para parsear cualquier CSV

Las comas dentro de comillas rompen soluciones ingenuas

pontIA

CSV válido

```
order_id,product,amount
1001,"Camiseta, azul",19.99
```

```
cut -d',' ...
```

Aquí interpretaría cuatro campos en vez de tres.

Para exploración rápida: head, grep, wc.
Para transformación fiable: Python, pandas o una herramienta de parsing.

5

Combinar comandos

Pipes y redirecciones convierten operaciones pequeñas en flujos reproducibles.

Todo proceso maneja tres flujos

La shell puede conectarlos, mostrarlos o guardarlos

pontIA

`stdin · 0`

entrada

`stdout · 1`

resultado

`stderr · 2`

errores

Un error no es 'texto rojo': es un flujo separado que podemos tratar explícitamente.

La pipe entrega stdout al siguiente comando

Lee la cadena de izquierda a derecha

pontIA

Composición

```
$ grep "ERROR" logs/pipeline.log | wc -l
3
```

1. grep selecciona líneas → **2.** | conecta la salida → **3.** wc cuenta

> reemplaza; >> añade

La diferencia es pequeña en sintaxis y grande en consecuencias

pontIA

>

Crea el archivo o reemplaza su contenido.

```
grep "ERROR" log > errors.txt
```

>>

Crea el archivo o añade al final.

```
echo "Fin" >> errors.txt
```

⚠ Antes de usar > sobre un archivo importante, comprueba si ya existe.

stderr y el código de salida explican si algo falló

Separa el mensaje del estado del proceso

pontIA

Salida y error

```
$ python3 scripts/process.py > output/run.txt 2> output/errors.txt
$ echo $?
1
```

0
éxito

≠0
fallo

`$?` solo conserva el estado del comando más reciente.

Construye la solución por capas

Prueba cada transformación antes de añadir la siguiente

pontIA

DE SIMPLE A COMPUESTO

1
```
$ grep "ERROR" logs/pipeline.log
```
Filtra

2
```
$ grep "ERROR" logs/pipeline.log | sort
```
Ordena

3
```
$ grep "ERROR" logs/pipeline.log | sort | uniq -c
```
Agrupa

pontIA

Un informe puede ser simple y reproducible

Combina reemplazo inicial y anexos controlados

pontIA

Informe

```
$ echo "RESUMEN DE ERRORES" > output/report.txt
$ grep "ERROR" logs/pipeline.log | sort | uniq -c >> output/report.txt
$ echo "TOTAL" >> output/report.txt
$ grep -c "ERROR" logs/pipeline.log >> output/report.txt
```

pontIA

La primera línea reinicia el informe; las siguientes añaden contenido.

6

Ejecutar y diagnosticar

Variables, permisos y procesos convierten una secuencia manual en una tarea ejecutable.

Una variable de entorno configura sin cambiar el código

export hace que los procesos hijos puedan leerla

pontIA

Configuración

```
$ export DATA_PATH="data/orders.csv"
$ echo $DATA_PATH
data/orders.csv

$ python3 scripts/process.py
```

NOMBRE

DATA_PATH

VALOR

data/orders.csv

⚠ No pongas espacios alrededor de `=`.

PATH es la lista de lugares donde la shell busca comandos

type o command -v explican qué se ejecutará

pontIA

Resolución de comandos

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin

$ type python3
python3 is /usr/bin/python3

$ command -v bash
/usr/bin/bash
```

command not found

Puede significar:

- error de escritura
- programa no instalado
- ejecutable fuera de PATH

Los permisos indican quién puede leer, escribir o ejecutar

Existen también diferentes tipos de propietarios

pontIA

clase	lectura (r)	escritura (w)	ejecución (x)
usuario (u)	leer contenido	modificar contenido	ejecutar archivo
grupo (g)	mismos bits, evaluados para el grupo		
otros (o)	última regla si no coinciden usuario ni grupo		

pontIA

ⓘ En directorios: r lista nombres · w crea/elimina entradas · x permite atravesar.

Los permisos indican quién puede leer, escribir o ejecutar

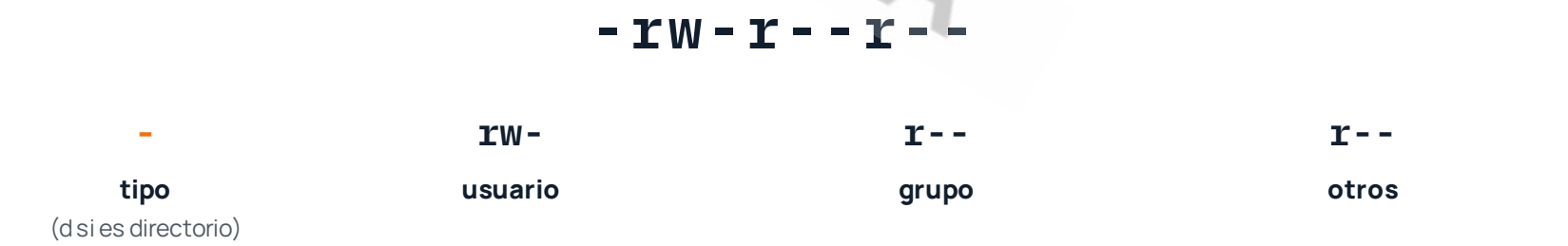
ls -l muestra el contrato del archivo

pontIA

Permisos

```
$ ls -l scripts/generate_report.sh
-rw-r--r--  1 alumno staff 842 Aug 11 generate_report.sh
```

pontIA



Los permisos indican quién puede leer, escribir o ejecutar

chmod permite cambiar los permisos de un usuario

pontIA

Permisos

```
$ chmod 754 scripts/generate_report.sh
$ ls -l scripts/generate_report.sh
-rwxr-xr--  1 alumno staff 842 Aug 11 generate_report.sh
```

Valor Octal	Binario	Permisos	Descripción
0	000	`---`	Ningún permiso
1	001	`--x`	Sólo ejecución
2	010	`-w-`	Sólo escritura
3	011	`-wx`	Escritura y ejecución
4	100	`r--`	Sólo lectura
5	101	`r-x`	Lectura y ejecución
6	110	`rw-`	Lectura y escritura
7	111	`rwx`	Lectura, escritura y ejecución

pontIA

Puedes ejecutar un script con la shell o hacerlo ejecutable

Son dos caminos relacionados, pero no idénticos

pontIA

Invocar la shell

```
bash scripts/generate_report.sh
```

No exige permiso x en el archivo

Ejecutar el archivo

```
chmod +x scripts/generate_report.sh
./scripts/generate_report.sh
```

Exige permiso x y un shebang válido (combinación específica de los caracteres ./)

pontIA

Python y Bash resuelven capas distintas del flujo

La terminal orquesta; Python procesa

pontIA

Ejecución

```
$ export DATA_PATH=data/orders.csv
$ python3 scripts/process.py
Procesados: 842 registros
Salida: output/summary.csv

$ bash scripts/generate_report.sh
Informe generado
```

Bash

Conecta pasos y comandos

Python

Aplica lógica y procesamiento robusto

Lanza y observa un proceso en segundo plano

El PID identifica el proceso; [1] identifica el trabajo de la shell

pontIA

Ejemplo

```
$ sleep 300 &
[1] 33342
$ ps -p 33342
  PID TTY           TIME CMD
33342 ttys002    0:00.00 sleep 300
$ jobs
[1]+ Running sleep 300 &
```

&

Lanza el comando en segundo plano y devuelve el prompt

`ps / jobs`

Muestran el PID, el estado y los trabajos de esta shell

Trae el trabajo al frente y termínalo con seguridad

Ctrl+Z lo pausa; fg y kill actúan sobre el trabajo %1

pontIA

Ejemplo

```
$ sleep 300 &
[1] 8421
$ fg %1
sleep 300
^Z
[1]+ Stopped sleep 300
$ kill %1
[1]+ Terminated sleep 300
```

`fg %1`

Trae el trabajo 1 al primer plano; Ctrl+Z lo vuelve a pausar

`kill %1`

Envía SIGTERM; usa kill -9 solo como último recurso

Un proceso puede observarse, detenerse o finalizar

Empieza por Ctrl+C; usa kill solo cuando conoces el PID

pontIA

`ps`	Procesos asociados a la sesión
`ps aux`	Vista más amplia de procesos
`top`	Actividad en tiempo real; q para salir
`Ctrl+C`	Interrumpe el proceso en primer plano
`kill PID`	Solicita terminar un proceso concreto

pontIA

Gracias



Sergio Benito Martín

www.pontia.tech pontia@sergiobenito.com

A

Bonus: utilidades complementarias

HTTP, compresión, almacenamiento y acceso remoto

curl trae recursos web desde la terminal

Empieza con una URL y añade opción solo cuando las necesitas

pontIA

Una petición mínima

```
$ curl -I https://example.com
```

comando
qué herramienta

opción
cómo pedirlo

URL
a dónde

curl hace peticiones HTTP

Estas opciones cubren inspección, redirecciones y guardado

pontIA

Opciones esenciales

```
$ curl -I https://example.com
$ curl -L https://example.com
$ curl -o pagina.html https://example.com
$ curl -s https://example.com
```

`-I`	Solo cabeceras
`-L`	Sigue redirecciones
`-o`	Guarda en archivo
`-s`	Modo silencioso

Construye una petición curl por capas

Comprueba la respuesta antes de guardar el resultado

pontIA

DE PETICIÓN A ARCHIVO

1 `$ curl https://api.github.com` **Pide**

2 `$ curl -I https://api.github.com` **Inspecciona**

3 `$ curl -o respuesta.json https://api.github.com` **Guarda**

pontIA

Chuleta básica de curl

Usa URLs de confianza y revisa el destino antes de guardar datos.

Ver contenido	`curl URL`
Cabeceras	`curl -I URL`
Redirecciones	`curl -L URL`
Guardar	`curl -o archivo URL`
Diagnosticar	`curl -v URL`

pontIA

tar agrupa y comprime archivos desde la terminal

Crear un paquete no elimina los originales; inspecciónalo antes de extraerlo

pontIA

Crear un archivo comprimido

```
$ tar -czf entrega.tar.gz proyecto/
```

comando
qué herramienta

opciones
cómo empaquetar

destino y origen
qué guardar

Chuleta básica de tar

Lista el contenido antes de extraer un archivo que no conoces.

Crear .tar.gz	`tar -czf backup.tar.gz carpeta/`
Listar sin extraer	`tar -tzf backup.tar.gz`
Extraer	`tar -xzf backup.tar.gz`
Elegir destino	`tar -xzf backup.tar.gz -C destino/`
Identificar tipo	`file backup.tar.gz`

pontIA

Linux/Bash archivos

Esquema comparativo de los archivos de configuración de shell bash

pontIA

→ Carga .bashrc*

Archivo	.profile	.bash_profile	.bashrc
Tipo de shell	Login (sh, bash)	Login (Solo bash)	Interactiva (No-Login)
Cuándo se ejecuta	Al iniciar sesión general en el shell	SSH, Consola TTY, MacOS (al autenticarse	Cada nueva pestaña o ventana de la terminal
Uso recomendado	Variables de entorno generales (ej. $PATH)	Variables específicas de Bash	Alias, funciones, personalización del prompt, colores

PontIA

Linux/Bash y macOS/zsh: mismos objetivos

Cambia el archivo que se carga, no el tipo de configuración que guardas.

pontIA

Linux / Bash

- Shell habitual: Bash
- Interactiva: ~/.bashrc
- Login: ~/.bash_profile o ~/.profile

macOS / zsh

- Shell habitual: zsh
- Interactiva: ~/.zshrc
- Login: ~/.zprofile

Aplicar los cambios

```
$ source ~/.bashrc
$ source ~/.zshrc
```

Archivos de inicio de Bash y zsh

Linux suele usar Bash; macOS usa zsh. Interactiva y login no son lo mismo.

Bash interactiva	~/.bashrc
Bash de login	~/.bash_profile
zsh interactiva	~/.zshrc
zsh de login	~/.zprofile
Fallback común	~/.profile

pontIA

Los archivos de inicio preparan tu shell

Son archivos de texto ocultos en tu home (~) que la shell lee al iniciar una sesión.

pontIA

La shell ejecuta su contenido al iniciar

```
alias ll='ls -lah'
export EDITOR=nano
export PATH="$HOME/bin:$PATH"
source ~/.zshrc
```

Alias	Atajos frecuentes
ENV	Configurar programas
PATH	Buscar ejecutables
Cargar	Cargar sin reiniciar

Qué configurar en estos archivos

Centraliza la personalización de la shell y evita guardar secretos.

pontIA

Alias	`alias ll='ls -lah'`
Variables	`export PROJECT_HOME=~/proyecto`
Añadir al PATH	`export PATH="$HOME/bin:$PATH"`
Aplicar cambios	`source ~/.zshrc`
Reutilizar	`source ~/.bashrc desde .bash_profile`

pontIA

Espacio en disco: df y du

Distingue la capacidad del sistema del tamaño ocupado por cada carpeta.

pontIA

Disco disponible	`df -h`
Tamaño de carpeta	`du -sh carpeta/`
Comparar carpetas	`du -sh ./*`
Ordenar por tamaño	`du -h carpeta/ | sort -h`
Vista rápida	`df -h && du -sh ./*`

pontIA

Acceso remoto: ssh y scp

Verifica el servidor y la ruta antes de conectarte o copiar archivos.

Conectar	`ssh usuario@servidor`
Subir archivo	`scp archivo usuario@servidor:/ruta/`
Descargar	`scp usuario@servidor:/ruta/archivo .`
Cerrar sesión	`exit`
Diagnóstico	`ssh -v usuario@servidor`

pontIA